tip:所有翻译均为机翻（部分影响理解的翻译错误做了修改）

Codeforces Round 990 (Div. 2)

暂时包含A-C

A. Alyona and a Square Jigsaw Puzzle

time limit per test:1 second

memory limit per test:256 megabytes

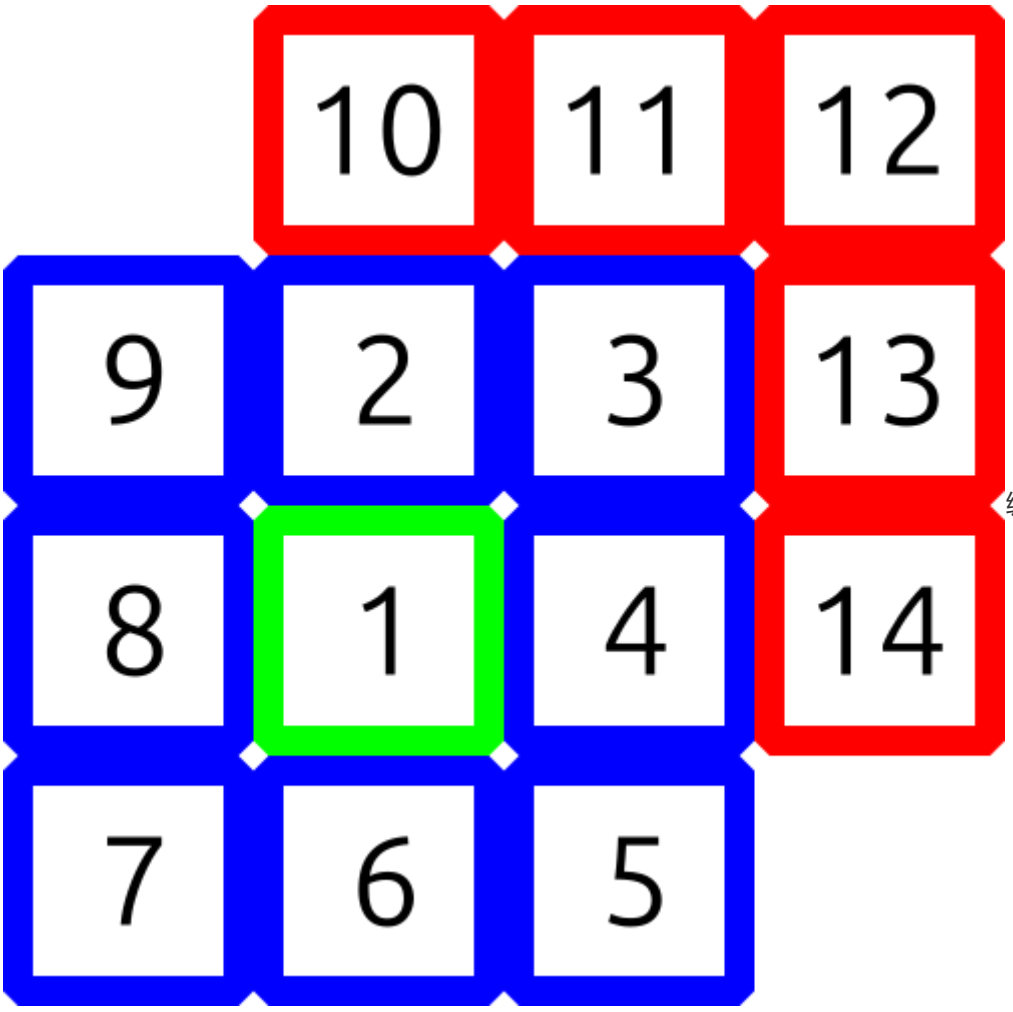
2047A

原题

Alyona assembles an unusual square Jigsaw Puzzle. She does so in N days in the following manner:

- On the first day, she starts by placing the central piece in the center of the table.
- On each day after the first one, she places a certain number of pieces around the central piece in clockwise order, always finishing each square layer completely before starting a new one.

For example, she places the first 14 pieces in the following order:



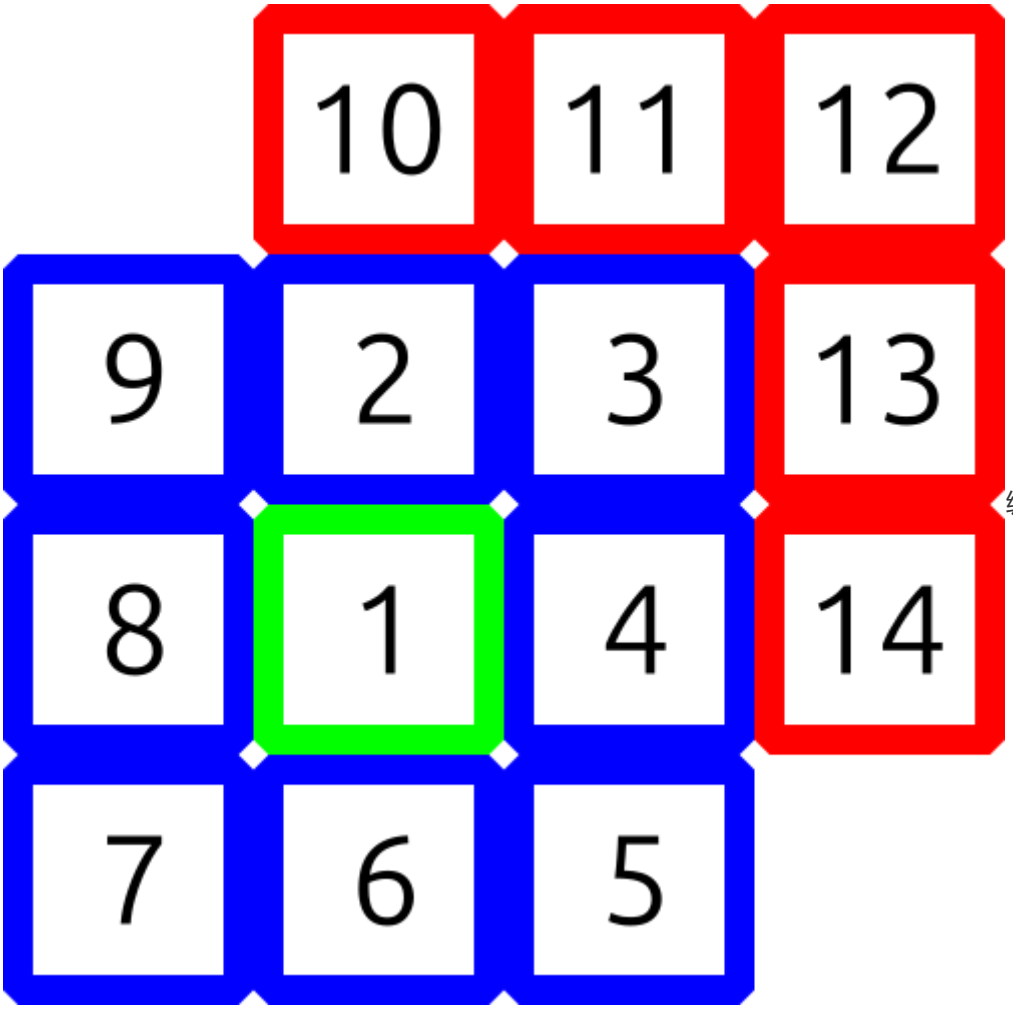
编辑The colors denote the layers. The third layer is still unfinished.

Alyona is happy if at the end of the day the assembled part of the puzzle does not have any started but unfinished layers. Given the number of pieces she assembles on each day, find the number of days Alyona is happy on.

阿廖娜组装了一个不寻常的方形拼图。她以下列方式在 n 天内完成此项工作：

-在第一天，她首先把中间的那块放在桌子的中央。
-在第一个之后的每一天，她将一定数量的碎片按顺时针顺序放置在中心碎片周围，总是在开始一个新的方块之前完成每个正方形图层。

例如，她将第一个 14块按以下顺序排列：



编辑颜色表示层。第三层还没有完成。

如果在一天结束时，谜题的组装部分没有任何开始但未完成的层，Alyona就会很高兴。给定她每天组装的零件的数量，找出Alyona快乐的天数。

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 500$). The description of the test cases follows.

The first line contains a single integer n ($1 \le n \le 100$), the number of days.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 100$, $a_1 = 1$), where a_i is the number of pieces Alyona assembles on the i-th day.

It is guaranteed in each test case that at the end of the n days, there are no unfinished layers.

每个测试包含多个测试用例。第一行包含测试用例的数量 t ($1 \le t \le 500$)。下面是测试用例的描述。

第一行包含一个整数 n ($1 \le n \le 100$)，表示天数。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 100$, $a_1 = 1$)，其中 a_i 是Alyona在第 i 天组装的碎片数。

在每个测试用例中，保证在 n 天结束时，没有未完成的层。

Output

For each test case, print a single integer: the number of days when Alyona is happy.

对于每个测试用例，打印一个整数：Alyona快乐的天数。

Example

Input

```
5
 1
 1
 2
 1 8
 5
 1 3 2 1 2
 7
 1 2 1 10 2 7 2
 14
 1 10 10 100 1 1 10 1 10 2 10 2 10 1
```

Output

```
1
 2
 2
 2
 3
```

题解

当放置过的方块的数量等于$(2n-1)^2, n \in N^*$时， 第n层刚好被填满， 快乐的天数加一。

代码如下

```
#include <bits/stdc++.h>
using namespace std;
```

```
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

void solve(){
    ll n = read();
    vector<ll> a(n, 0);
    for(ll i=0;i<n;i++){
        a[i] = read();
    }
    ll ans = 0;
    ll cnt = 0;
    ll need = (2*cnt+1)*(2*cnt+1);
    ll did = 0;
    for(ll i=0;i<n;i++){
        did += a[i];
        while (did>=need){
            if (did == need){
                ans ++;
            }
            cnt += 1;
            need = (2*cnt+1)*(2*cnt+1);
        }
    }
    print(ans);
}


int main(){
    ll t= read();
    while (t--){
        solve();
    }
}
```

B. Replace Character

time limit per test:1 second

memory limit per test:256 megabytes

2047B

原题

You're given a string s of length n, consisting of only lowercase English letters.

You must do the following operation exactly once:

- Choose any two indices i and j ($1 \le i, j \le n$). You can choose $i = j$.
- Set $s_i := s_j$.

You need to minimize the number of distinct permutations† of s. Output any string with the smallest number of distinct permutations after performing **exactly one** operation.

† A permutation of the string is an arrangement of its characters into any order. For example, "bac" is a permutation of "abc" but "bcc" is not.

给您一个长度为 n 的字符串 s ，仅由小写英文字母组成。

以下操作必须精确执行一次：

—选择任意两个索引 i 和 j ($1 \le i, j \le n$)。您可以选择 $i = j$。 $s_i := s_j$（将s_i更改为s_j）。

您需要最小化 s 的不同排列 † 的数量。在执行**一次**操作后，输出具有最少不同排列数的任何字符串。

† 字符串的排列是将字符串中的字符按任意顺序排列。例如，“bac”是“abc”的排列，但“bcc”不是。

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 500$). The description of the test cases follows.

The first line of each test case contains n ($1 \le n \le 10$) — the length of string s.

The second line of each test case contains s of length n. The string contains only lowercase English letters.

每个测试包含多个测试用例。第一行包含测试用例的数量t ($1 \le t \le 500$)。下面是测试用例的描述。

每个测试用例的第一行包含字符串 s 的长度 n ($1 \le n \le 10$) 。

每个测试用例的第二行包含长度为 n 的 s$ 。字符串中只包含小写英文字母。

Output

For each test case, output the required s after applying exactly one operation. If there are multiple solutions, print any of them.

对于每个测试用例，在应用一个操作后输出所需的 s 。如果有多个解决方案，打印其中任何一个。

Example

Input

```
6
 3
 abc
 4
 xyyx
 8
 alphabet
 1
 k
 10
 aabbccddee
 6
 ttbddq
```

Output

```
cbc
 yyyx
 alphaaet
 k
 eabbccddee
 tttddq
```

题解

字符串的排列数量不收其字符顺序的影响。

设某个字符串（长度为n）中有 a_1个x_1字符....a_m个x_m字符

则排列的数量为 $\frac{A_n^n}{\prod_{j=1}^{m} A_{a_j}^{a_j}}$

不妨考虑字符串$aaaabbbccd$，初始排列数量为 $\frac{A_{10}^{10}}{A_4^4 \times A_3^3 \times A_2^2 \times A_1^1} = 12600$，在更改的过程中分子不会变化，因此我们需要让分母尽可能大。

不难发现， 将出现次数最少的d换为出现次数最多的a，排列数量为 $\frac{A_{10}^{10}}{A_5^5 \times A_3^3 \times A_2^2} = 2520$。

我们只需找到出现次数最多的字符，用它去替换一个出现次数最少的字符。

代码如下

python

```python
t = int(input())
for _ in range(t):
    n = int(input())
    s = input()
    m = {}
    for i in s:
         m[i] = m.get(i, 0) + 1
    values = sorted([(i, m[i]) for i in m],key=lambda x:x[1], reverse=True)
    print(s.replace(values[-1][0], values[0][0], 1))
```

c++

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<long long,long long> PLL;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

bool cmp(PLL x, PLL y){
    return x.second > y.second;
}
void solve(){
    ll n = read();
    char s[20];
    map<ll, ll> m;
    for(ll i=0;i<n;i++){
        s[i] = getchar();
        m[s[i]] ++ ;
    }
    vector<PLL> values;
    for(auto pair:m){
        values.push_back(pair);
    }
    ll len = values.size();
    sort(values.begin(), values.end(), cmp);
    for(ll i=0;i<n;i++){
        if (s[i] == values[len-1].first){
            s[i] = values[0].first;
            break;
        }
    }
    for(ll i=0;i<n;i++){
        putchar(s[i]);
    }
    putchar('\n');
```

```
}

int main(){
    ll t = read();
    while(t--){
        solve();
    }
}
```

C. Swap Columns and Find a Path

time limit per test:2 second

memory limit per test:512 megabytes

2047C

原题

There is a matrix consisting of 2 rows and n columns. The rows are numbered from 1 to 2 from top to bottom; the columns are numbered from 1 to n from left to right. Let's denote the cell on the intersection of the i-th row and the j-th column as (i,j). Each cell contains an integer; initially, the integer in the cell (i,j) is $a_{i,j}$.

You can perform the following operation any number of times (possibly zero):

- choose two columns and swap them (i. e. choose two integrs x and y such that $1 \le x < y \le n$, then swap $a_{1,x}$ with , and then swap $a_{2,x}$ with $a_{2,y}$).

After performing the operations, you have to choose a path from the cell $(1,1)$ to the cell $(2,n)$. For every cell (i,j) in the path except for the last, the next cell should be either $(i1,j)$ or $(i,j1)$. Obviously, the path cannot go outside the matrix.

The cost of the path is the sum of all integers in all $(n1)$ cells belonging to the path. You have to perform the operations and choose a path so that its cost is **maximum** possible.

有一个由 2 行和 n 列组成的矩阵。行从上到下编号为 1 到 2；列从左到右编号为 1 到 n 。我们将位于第i 行和 第j 列交点上的单元格表示为 (i,j)。每个单元格包含一个整数；最初，单元格 (i,j) 中的整数是 $a_{i,j}$ 。

您可以执行以下操作任意次数(可能为零)：

-选择两列并交换它们(即：选择两个整数 x 和 y 使得 $1 \le x < y \le n$ ，然后将 $a_{1,x}$ 与 $a_{1,y}$交换，然后将 $a_{2,x}$ 与 $a_{2,y}$ 交换)。

操作完成后，需要选择从单元格 $(1,1)$ 到单元格 $(2,n)$ 的路径。对于路径中除最后一个单元格之外的每个单元格 (i,j) ，下一个单元格应该是 $(i1,j)$ 或 $(i,j1)$。很明显，路径不能走出矩阵。

路径的代价是属于该路径的所有 $(n1)$ 单元格中所有整数的和。你必须执行操作并选择一条路径，使它的代价是**最大**的。

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 5000$). The description of the test cases follows.

Each test case consists of three lines:

- the first line contains one integer n ($1 \le n \le 5000$)— the number of columns in the matrix;

- the second line contains n integers $a_{1,1}, a_{1,2}, \ldots, a_{1,n}$ ($-10^5 \le a_{i,j} \le 10^5$) — the first row of the matrix;

- the third line contains n integers $a_{2,1}, a_{2,2}, \ldots, a_{2,n}$ ($-10^5 \le a_{i,j} \le 10^5$) — the second row of the matrix.

It is guaranteed that the sum of n over all test cases does not exceed 5000.

每个测试包含多个测试用例。第一行包含测试用例的数量 t ($1 \le t \le 5000$)。下面是测试用例的描述。

每个测试用例由三行组成：

-第一行包含一个整数 n ($1 \le n \le 5000$)-矩阵中的列数；
-第二行包含 n 个整数$a_{1,1}, a_{1,2}, \ldots, a_{1,n}$ ($-10^5 \le a_{i,j} \le 10^5$)-矩阵的第一行；
-第三行包含 n个整数 $a_{2,1}, a_{2,2}, \ldots, a_{2,n}$ ($-10^5 \le a_{i,j} \le 10^5$) -矩阵的第二行。

保证所有测试用例 n 的和不超过 5000 。

Output

For each test case, print one integer — the maximum cost of a path you can obtain.

对于每个测试用例，打印一个整数—您可以获得的路径的最大代价。

Example

Input

```
3
 1
 -10
 5
 3
 1 2 3
 10 -5 -3
 4
 2 8 5 3
 1 10 3 4
```

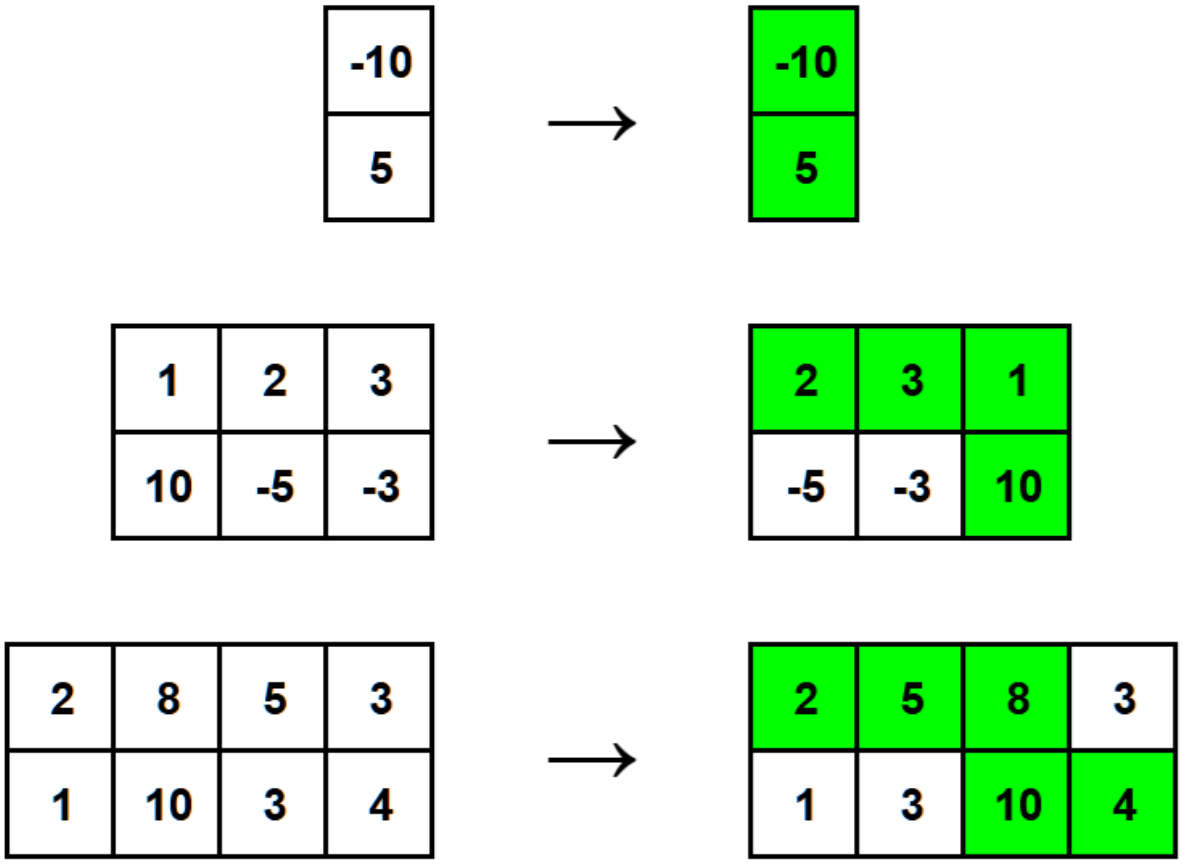
Output

-5
16
29

Note

Here are the explanations of the first three test cases of the example. The left matrix is the matrix given in the input, the right one is the state of the matrix after several column swaps (possibly zero). The optimal path is highlighted in green.

下面是对该示例的前三个测试用例的解释。左边的矩阵是输入中给出的矩阵，右边的矩阵是经过几列交换(可能为零)后的矩阵状态。最优路径用绿色突出显示。



编辑

题解

考虑到只能向下和向右走，可将整个路径分为 $a_{1,1} \to a_{1,m}, a_{1,m} \to a_{2,m}, a_{2,m} \to a_{2,n}$三部分

因为可以无限次交换，我们一定可以将$a_{1,j} > a_{2,j}$的列放到左边，$a_{1,j} \leqslant a_{2,j}$ 的列放到右边

即我们一定可以构造出一种情况，使得路径能够取到每一列中的最大值。

考虑从第一行到第二行的过程：我们可以选择一列中较小数最大的那一列，将它放在中间。

这样构造出来的即为所求路径。

（答案实际上为$\sum_{t=1}^{n} max(a_{1,t}, a_{2,t}) max(\{min(a_{1,t}, a_{2,t}), t \in [1, n]\})$）

代码如下

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll INF = INT_MAX;


inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}


void solve(){
    ll n = read();
    ll a[3][5009];
    for(ll x=0;x<=1;x++){
        for(ll y=0;y<n;y++){
            a[x][y] = read();
        }
    }
    ll total =0;
    ll m = -INF;
    for(ll i=0;i<n;i++){
        total += max(a[0][i], a[1][i]);
        m = max(m, min(a[0][i], a[1][i]));
    }
    print(total + m);
}



int main(){
    ll t = read();
    while(t--){
        solve();
    }
}
```